TRƯỜNG ĐẠI HỌC THỦY LỢI — KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO NGHIỆM THU BÀI THỰC HÀNH 3

Đề tài: Hướng Dẫn Từng Bước (Step-by-Step) Triển Khai CI/CD Và Đóng Gói Docker Image Public Lên Docker Hub Cho Bài Toán AI

Sinh viên: Nguyễn Thuận Hóa • MSV: 2351260652 • Lớp: 65TTNT (Chuyên đề TTNT & KHDL)

BẢNG TỔNG HỢP CÁC ĐƯỜNG DẪN NGHIỆM THU CI/CD & DOCKER HUB

Hạng mục yêu cầu	Đường dẫn chính thức (URL)	Minh chứng hoàn thành
1. Triển khai CI/CD cho AI	github.com/ThuanHoa1110/ChuyenDe/actions Workflow: .github/workflows/ci.yml	54 tests Pytest tự động, lint flake8, docker build test, ghim action commit SHA, CPU offline
2. Docker Image Public	hub.docker.com/r/hanguynthun/hanoi-temperature Tag: `v1.0.0` • Digest: `sha256:d57f893a...`	Container non-root (`appuser`), checkpoint runtime mount, test API suy luận thành công

PHẦN 1: TRIỂN KHAI CI/CD CHO BÀI TOÁN AI (STEP-BY-STEP)

Bước 1.1: Phân tích Sự Khác biệt Cốt lõi của CI/CD cho Bài toán AI:
Khác với phần mềm thông thường (chỉ kiểm tra linting cú pháp hoặc logic CRUD cơ bản), CI/CD trong bài toán Trí tuệ Nhân tạo (MLOps Continuous Integration) đòi hỏi quy trình tự động hóa khắt khe ở 3 cấp độ:
1. Kiểm tra Dữ liệu (Data Testing): Đảm bảo tính toàn vẹn của chuỗi thời gian khí tượng, đúng chu kỳ 5 phút, không rò rỉ tương lai và tỷ lệ khuyết thiếu 0% NaN.
2. Kiểm tra Mô hình (Model Testing): Đảm bảo forward tensor shape, tính toán loss function và backward gradient flow không bị NaN/Inf.
3. Kiểm tra Suy luận & Phục vụ (Serving Testing): Nạp checkpoint và scaler thực tế, phản hồi API đúng cấu trúc với độ trễ < 100ms.

Bước 1.2: Xây dựng Bộ Kiểm thử Toàn diện 54 Tests Pytest:
Nhóm đã xây dựng bộ kiểm thử tự động gồm 10 tệp test độc lập phủ kín toàn bộ pipeline:
• `test_smoke.py` (6 tests): Kiểm tra siêu dữ liệu gói, cấu trúc thư mục, tệp cấu hình YAML.
• `test_validation.py` (6 tests): Bắt chính xác các lỗi cố ý: trùng timestamp, nhiệt độ 150°C, đảo ngược thời gian, trôi đồng hồ.
• `test_window.py` (5 tests): Kiểm tra chia mốc thời gian (Train 70%, Val 15%, Test 15%), chứng minh 100% không rò rỉ tương lai.
• `test_baseline.py` (4 tests): Kiểm tra tính tái lập của Persistence, Ridge, LightGBM.
• `test_image.py` (5 tests): Kiểm tra mã hóa ảnh 3 kênh (Normalized, Past Deltas, Observation Mask).
• `test_dinov3.py` (4 tests): Kiểm tra forward batch 1/batch 2, backward chỉ cập nhật Regression Head (frozen backbone), save/load checkpoint.
• `test_api.py` (11 tests): Kiểm tra endpoint `/health`, `/metadata`, payload hợp lệ và 6 trường hợp payload sai cấu trúc (HTTP 422).
• `test_docker.py` (4 tests): Kiểm tra Dockerfile non-root, base image pinned, .dockerignore.
• `test_cicd.py` (5 tests): Kiểm tra cú pháp YAML của workflow, quyền hạn bảo mật và tính toàn vẹn của tài liệu PDF.
• `test_release.py` (4 tests): Kiểm tra gói dữ liệu Kaggle release.

Bước 1.3: Cấu hình Workflow Tự động Hóa (`.github/workflows/ci.yml`):
Workflow CI được kích hoạt tự động mỗi khi có `pull_request` hoặc `push` vào nhánh `main`:
1. Checkout Source Code: Sử dụng `actions/checkout@v4` ghim commit SHA đầy đủ.
2. Setup Python & Caching: Cài đặt Python 3.12, tận dụng pip cache theo mã băm `requirements.lock`.
3. Linting: Kiểm tra chất lượng mã nguồn bằng `flake8`.
4. Chạy 54 Tests Pytest: Toàn bộ bài test chạy hoàn toàn offline trên CPU (không cần GPU, không tải checkpoint gated

từ Hugging Face), bảo đảm CI luôn ổn định 100%.
5. Docker Build Dry-Run: Biên dịch thử nghiệm Docker image trên runner CI để phát hiện sớm lỗi đóng gói.

Bước 1.4: Minh chứng Thực tế Chạy CI (Kịch bản CI Đỏ & CI Xanh):
• Kịch bản CI Đỏ (Red Run): Cố ý đưa vào kiểm thử một assertion sai (ví dụ nhiệt độ sai phân không hợp lệ). Hệ thống GitHub Actions lập tức đánh dấu thất bại (Red X), chặn đứng việc merge code lỗi.
• Kịch bản CI Xanh (Green Run): Sau khi kiểm tra toàn bộ 54 test cases đều thỏa mãn, job CI chuyển trạng thái xanh toàn bộ (Green Checkmark) trong vòng 99.49 giây.

PHẦN 2: BUILD DOCKER IMAGE CHO MODEL AI & PUBLIC LÊN DOCKER HUB (STEP-BY-STEP)

Bước 2.1: Thiết kế Dockerfile Chuẩn Bảo mật Công nghiệp:
Tệp `Dockerfile` được tối ưu hóa theo các nguyên tắc an ninh phần mềm:
• Base Image Pinned: Sử dụng `python:3.12-slim` ghim phiên bản cố định để triệt tiêu lỗ hổng bảo mật và giảm dung lượng image.
• Tài khoản Non-Root (`appuser`): Tạo user riêng với `UID 10001`, tuyệt đối không chạy ứng dụng dưới quyền `root`.
• Tối ưu Hóa Build Context: Tệp `.dockerignore` loại trừ toàn bộ dữ liệu thô (`data/raw/`, `data/private/`), file môi trường `.env`, cache và checkpoints.
• Cơ chế Runtime Volume Mount: Checkpoint mô hình không bị nướng cứng vào image mà được gắn ngoài lúc chạy container (`-v /checkpoints:/app/checkpoints:ro`), tuân thủ điều khoản Meta DINOv3.

```
Bước 2.2: Quy trình Build Docker Image Cục bộ:
Thực thi lệnh xây dựng image tại thư mục gốc dự án:
    `docker build -t hanguynthun/hanoi-temperature:v1.0.0 .`
Quá trình build diễn ra nhiều lớp (multi-layer caching), hoàn thành trong chưa đầy 3 phút với kích thước tối ưu (~500 MB).
```

Bước 2.3: Tự động Hóa Xuất bản Lên Docker Hub (CD Pipeline):
Workflow `.github/workflows/release.yml` được kích hoạt tự động mỗi khi có tag phát hành `v*`:
1. Xác thực an toàn với Docker Hub thông qua 2 bí mật: `DOCKERHUB_USERNAME` và `DOCKERHUB_TOKEN` (được che bảo mật trong log).
2. Tự động gắn tag SemVer `v1.0.0` và `latest`.
3. Đẩy image lên Docker Hub Public Registry và xuất mã băm bất biến (Image Digest) vào phần tóm tắt của workflow.

Bước 2.4: Thông tin Image Chính thức trên Docker Hub:
• Public URL: https://hub.docker.com/r/hanguynthun/hanoi-temperature
• Image Digest: `sha256:d57f893a7499691bce9857d904b779a5e8e8203fcf982e5ee600b3f545a987d6`
• Trạng thái: Công khai (Public), bất kỳ ai cũng có thể kéo về sử dụng ngay.

Bước 2.5: Kiểm định Kéo & Vận hành Trên Máy Sạch Độc lập (Clean Host Run):
Người chấm điểm hoặc hội đồng chỉ cần thực hiện 2 lệnh duy nhất trên terminal máy sạch:
1. Kéo image chính thức từ Docker Hub:
   `docker pull hanguynthun/hanoi-temperature:v1.0.0`
2. Khởi chạy container với checkpoint runtime mount:
   `docker run -d --name weather-api -p 8000:8000 -v ${PWD}/checkpoints:/app/checkpoints:ro hanguynthun/hanoi-temperature:v1.0.0`
3. Kiểm tra sức khỏe hệ thống:
   `curl http://localhost:8000/health` ==> Trả về `{'status': 'healthy', 'model_version': '1.0.0'}`.
4. Gọi suy luận dự báo nhiệt độ 6 trạm:
   `curl -X POST http://localhost:8000/predict -H 'Content-Type: application/json' -d @docs/api_example.json`
   Kết quả: Trả về đúng 6 nhiệt độ dự báo liên tục sau 60 phút, độ trễ suy luận < 30ms.

Bước 2.6: Quy trình Rollback Phiên bản Bất biến:
Trong trường hợp phiên bản mới phát sinh sự cố, quản trị viên có thể rollback về phiên bản ổn định ngay lập tức bằng cách khởi chạy trực tiếp thông qua Image Digest bất biến:
   `docker run -d -p 8000:8000 hanguynthun/hanoi-temperature@sha256:d57f893a...`
Cơ chế này bảo đảm hệ thống phục hồi dịch vụ trong vòng dưới 5 giây mà không làm gián đoạn người dùng.

KẾT LUẬN NGHIỆM THU: Toàn bộ quy trình CI/CD cho bài toán AI và đóng gói xuất bản Docker Image lên Docker Hub đã được thực hiện step-by-step đầy đủ, có tài liệu hướng dẫn và mã nguồn kiểm chứng tự động 100% theo đúng yêu cầu của Giảng viên.